CLASE 2 · PROGRAMACIÓN EN BASH

```
usuario@clase:~$ grep | sed | awk
```

Procesar texto en el shell

Expresiones regulares, sed y awk

Basado en los capítulos 4 a 6 de *Bash Guide for Beginners*, de Machtelt Garrels

CAPÍTULO 4

Expresiones regulares

Patrones que describen conjuntos de cadenas, y grep para encontrarlos.

04

4.1 · QUÉ SON

¿Qué es una expresión regular?

Un patrón que describe un **conjunto de cadenas**.

Se construyen como las expresiones aritméticas: combinando expresiones pequeñas con operadores.

El patrón

`c.*h`

coincide con

`cash` `catch` `cheetah` `coach`

Caracteres comunes
Letras y dígitos coinciden consigo mismos.

Metacaracteres
Tienen significado especial; con \ delante se toman literalmente.

Combinar
Concatenar, alternar con | y agrupar con ( ). Precedencia: repetición, concatenación, alternancia.

4.1.2 · METACARACTERES

Los metacaracteres

Repetición

`.`	Cualquier carácter
`?`	Lo anterior, cero o una vez
`*`	Lo anterior, cero o más veces
`+`	Lo anterior, una o más veces
`{N}`	Exactamente N veces
`{N,}`	N veces o más
`{N,M}`	Entre N y M veces

Anclas y listas

`^`	Inicio de línea
`$`	Fin de línea
`[abc]`	Uno de los caracteres de la lista
`[^abc]`	Uno que NO esté en la lista
`[a-z]`	Uno del rango

4.2 · GREP

grep: encontrar líneas

```
$ grep root /etc/passwd
root:x:0:0:root:/root:/bin/bash
operator:x:11:0:operator:/root:/sbin/nologin
$ grep -n root /etc/passwd
1:root:x:0:0:root:/root:/bin/bash
12:operator:x:11:0:operator:/root:/sbin/nologin
$ grep -v bash /etc/passwd | grep -v nologin
$ grep -c false /etc/passwd
7
```

`-n`	Muestra el número de línea
`-v`	Invierte: líneas que NO coinciden
`-c`	Cuenta las líneas que coinciden
`-i`	Ignora mayúsculas y minúsculas
`-w`	Solo palabras completas
`-F`	Cadena literal, sin regex
`-E`	Expresiones extendidas

4.2.2.1 · ANCLAS

Anclar el patrón: inicio, fin, palabra

Empieza con…

```
$ grep ^root /etc/passwd
```

Solo líneas que comienzan con root.

Termina con…

```
$ grep :$ /etc/passwd
```

Cuentas sin shell asignada: la línea termina en «:».

Inicio de palabra

```
$ grep export ~/.bashrc | grep '\<PATH'
```

PATH, pero no MANPATH ni otras rutas.

Palabra completa

```
$ grep -w / /etc/fstab
```

La partición raíz: / como palabra aislada.

Listas, comodines y comillas

Uno de la lista

```
$ grep '[yf]' /etc/group
sys:x:3:root,bin,adm
floppy:x:19:
```

Cinco letras, de c a h

```
$ grep '\<c...h\>' /usr/share/dict/words
catch  clash  cloth  coach  couch
```

Cualquier cantidad

```
$ grep '\<c.*h\>' /usr/share/dict/words
caliph  cash  catch  cheetah  …
```

Un asterisco literal

```
$ grep * /etc/profile
#  (nada: Bash expandió el *)
$ grep '*' /etc/profile
for i in /etc/profile.d/*.sh ; do
```

Siempre entre comillas simples: así Bash no toca el patrón antes de que llegue a grep.

Patrones de Bash no son regex

Significado	Bash (comodín)	Regex
Cualquier cadena	`*`	`.*`
Un carácter	`?`	`.`
Uno de la lista	`[abc]`	`[abc]`
Uno del rango	`[a-c]`	`[a-c]`
Uno fuera de la lista	`[!abc]` `[^abc]`	`[^abc]`
Clase POSIX	`[[:digit:]]`	`[[:digit:]]`

```
$ ls -ld [a-cx-z]*
app-defaults/  arabic/  bin/
crossover/  xml/
$ ls -ld [[:upper:]]*
Nautilus/  Schedule.sdc
```

Clases: alnum, alpha, digit, lower, upper, space, punct, xdigit…

CAPÍTULO 5

El editor de flujo sed

Transformar texto que llega de un archivo o de una tubería, sin abrir un editor.

05

5.1 · QUÉ ES SED

sed: un editor de flujo

Archivo o tubería
el texto de entrada

→

sed 'comandos'
transforma línea por línea

→

Salida estándar
redirigible con > archivo

Comandos

`p`	Imprimir	`d`	Borrar
`s`	Buscar y reemplazar	`a\`	Agregar texto debajo
`i\`	Insertar texto arriba	`c\`	Cambiar la línea
`r`	Leer un archivo	`w`	Escribir a un archivo

Opciones

`-n`	Modo silencioso
`-e`	Agregar un comando
`-f`	Leer comandos de un archivo

5.2.1 · IMPRIMIR

Imprimir las líneas que coinciden

El archivo ejemplo

```
1  Esta es la primera línea del ejemplo.
2  Es un texto con erores.
3  Muchos erores.
4  Tantos erores, todos estos erores me cansan.
5  Esta es una línea sin ningún error.
6  Esta es la última línea.
```

Solo las coincidencias

```
$ sed -n '/erores/p' ejemplo
Es un texto con erores.
Muchos erores.
Tantos erores, todos estos…
```

Sin **-n**, sed imprime todo el archivo y repite las líneas que coinciden.

5.2.2 · BORRAR Y RANGOS

Borrar líneas y trabajar con rangos

```
$ sed '/erores/d' ejemplo
```

Borra las líneas con «erores»: quedan la 1, la 5 y la 6.

```
$ sed '2,4d' ejemplo
```

Borra las líneas 2 a 4: el mismo resultado, por número.

```
$ sed '3,$d' ejemplo
```

Borra desde la 3 hasta el final ($): quedan las dos primeras.

```
$ sed -n '/un texto/,/Esta/p' ejemplo
```

Imprime desde la línea con «un texto» hasta la siguiente con «Esta»: líneas 2 a 5.

```
$ sed -n '/^Esta.*error\.$/p' ejemplo
Esta es una línea sin ningún error.
```

5.2.4 · BUSCAR Y REEMPLAZAR

Reemplazar con s/viejo/nuevo/

`s/erores/errores/`

Reemplaza solo la **primera** coincidencia de cada línea:

```
Tantos errores, todos estos erores me cansan.
```

`s/erores/errores/g`

Con **g** (global), todas las de la línea:

```
Tantos errores, todos estos errores me cansan.
```

CAPÍTULO 6

El lenguaje awk

Leer datos en columnas, filtrarlos, darles formato y hacer cálculos.

06

6.1 · QUÉ ES AWK

awk: describir los datos, no los pasos

Un programa awk es una serie de **reglas**: un patrón a buscar y una acción a realizar cuando aparece.

```
awk 'patrón { acción }' archivos
awk -f programa.awk archivos
```

Guiado por los datos

Se describe qué datos interesan y qué hacer con ellos, en lugar de cada paso como en un lenguaje procedural.

¿Por qué «awk»?

Por las iniciales de sus autores, en los años 70: Aho, Weinberger y Kernighan.

6.2.1 · CAMPOS

Cada línea se divide en campos

`$0` — la línea completa

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/hda1	121M	9.4M	105M	9%	/boot
$1	$2	$3	$4	$5	$6

Separador por defecto: uno o más espacios o tabulaciones.

```
$ df -h | awk '{ print $6, $5 }'
```

6.2.2 · PRINT Y FORMATO

print: elegir y ordenar columnas

Sin coma: se pegan

```
$ ls -l | awk '{ print $5 $9 }'
160orig
121script.sed
```

Con coma: separador de salida

```
$ ls -l | awk '{ print $5, $9 }'
160 orig
121 script.sed
```

Con texto y tabulaciones

```
$ df -h | sort -rnk 5 | head -3 | \
  awk '{ print "Partición " $6 "\t: " $5 " llena" }'
Partición /var    : 86% llena
Partición /usr    : 85% llena
Partición /home   : 70% llena
```

Secuencias especiales: **\t** tabulación, **\n** salto de línea, **\a** campana.

6.2.3 · PATRONES

Filtrar con expresiones regulares

Una regex entre barras funciona como patrón: la acción se aplica solo a las líneas que coinciden.

```
$ df -h | awk '/dev\/hd/ { print $6 "\t: " $5 }'
/       : 46%
/boot   : 10%
/usr    : 97%
```

```
$ ls -l /etc | awk '/\<(a|x).*\.conf$/ { print $9 }'
amd.conf
xinetd.conf
```

La barra dentro del patrón se escapa (\/). El primer punto significa «cualquier carácter»; el segundo, escapado, es un punto literal.

6.2.4 · PATRONES ESPECIALES

BEGIN y END

BEGIN { ... }
una vez, antes de leer la entrada

→

/patrón/ { ... }
por cada línea que coincide

→

END { ... }
una vez, al terminar

diskrep.awk

```
BEGIN { print "*** ATENCIÓN ***" }
/\<[89][0-9]%/ { print "Partición " $6 "\t: " $5 " llena" }
END { print "*** Hay que comprar discos ***" }
```

```
$ df -h | awk -f diskrep.awk
*** ATENCIÓN ***
Partición /usr    : 97% llena
*** Hay que comprar discos ***
```

6.3.1 · SEPARADOR DE CAMPOS

FS: el separador de entrada

```
$ awk 'BEGIN { FS=":" } { print $1 "\t" $5 }' /etc/passwd
kelly   Kelly Smith
franky  Franky B.
cathy   Catherine the Great
```

Se fija en BEGIN

Así la primera línea ya se lee con el separador correcto. Atajo equivalente: **awk -F:**

Elegirlo con cuidado

Con FS=",", «Sandy L. Wong, PhD, 64 Zoo St.» corre todos los campos un lugar.

6.3.2 · VARIABLES INTERNAS

OFS, ORS y NR

FS	Separador de campos de entrada
OFS	Separador de campos de salida (espacio)
ORS	Separador de registros de salida (\n)
NR	Número de registros leídos hasta ahora

procesados.awk

```
BEGIN { OFS="-" }
{ print "Registro " NR ": " $1, $2 }
END { print "Total: " NR }
```

```
$ awk -f procesados.awk prueba
Registro 1: record1-data1
Registro 2: record2-data2
Total: 2
```

Variables propias: sumar una columna

ingresos

```
20021009  20021013  consultancy  BigComp    2500
20021015  20021020  training     EduComp    2000
20021112  20021123  appdev       SmartComp  10000
20021204  20021215  training     EduComp    5000
```

total.awk

```
{ total = total + $5 }
{ print "Facturar " $5 " a " $4 }
END { print "Ingreso total: " total }
```

```
$ awk -f total.awk ingresos
Facturar 2500 a BigComp
Facturar 2000 a EduComp
Facturar 10000 a SmartComp
Facturar 5000 a EduComp
Ingreso total: 19500
```

Una variable nueva empieza vacía (vale 0 en una suma).

También se admite **total += $5**.